

Windows User and Permission Management Lab

Course: Operating Systems and You: Becoming a Power User

Module: Users, Groups and Permissions

Date: 20/08/2026

Author: Florencia Horita

1. Introduction

This lab aims to apply user, group, and permission management concepts in Windows using PowerShell.

A scenario was created to simulate basic resource administration within an organization. Different local user accounts and groups were configured, and NTFS permissions were subsequently assigned to a directory structure.

During the lab, we practiced creating and managing local users and groups, assigning group memberships, inspecting Access Control Lists (ACLs), configuring permissions using `icacls`, and understanding how permission inheritance works between directories.

The main objective was to understand the relationship between users, groups, resources, and permissions by using an administration model similar to those found in real-world Windows environments.

2. Environment

The lab was performed using the following environment:

- Virtualized Windows 11 Pro
- PowerShell
- NTFS file system
- macOS host
- Windows virtual machine

Administrative operations were performed from PowerShell with the privileges required to create local accounts and groups and modify file system permissions.

3. Lab Scenario

To simulate a work environment, three local users were defined:

- **Ana** — user belonging to the lab's general user group.
- **Carlos** — user belonging to the project team.
- **AdminLab** — user belonging to the lab's administrative group.

Three local groups were also defined:

- **LabUsers**
- **ProjectTeam**

- LabAdmins

The relationship between users and groups used during the lab was:

Ana → LabUsers

Carlos → ProjectTeam

AdminLab → LabAdmins

These groups were later used to manage access to different file system resources.

Using groups makes it possible to assign permissions according to functions or roles, avoiding the need to configure each user individually.

4. Creating Local Users

Local accounts were created from PowerShell using `New-LocalUser`.

To enter passwords without writing them directly as visible text in the commands, `Read-Host` was used together with the `-AsSecureString` parameter.

Commands used

```
$password = Read-Host "Enter password for Ana"
-AsSecureString
New-LocalUser -Name "Ana" -Password $password
```

The creation of the account was then verified using:

```
Get-LocalUser -Name "Ana"
```

The result showed `Enabled = True`, indicating that the account was enabled.

The same procedure was subsequently used to create the `Carlos` and `AdminLab` accounts.

Concepts applied

- Creating local users
- Password management using `SecureString`
- Verifying local accounts
- Enabled or disabled account status
- Difference between an enabled account and administrative privileges

Figure 1 — Creation and verification of the local user Ana

```
PS C:\WINDOWS\system32> $password = Read-Host "Enter password for Ana" -AsSecureString
Enter password for Ana: ***
PS C:\WINDOWS\system32> New-LocalUser -Name "Ana" -Password $password

Name Enabled Description
----
Ana   True
```

PS C:\WINDOWS\system32>

5. Creating Local Groups

Three local groups were created to organize users according to their functions within the scenario.

LabUsers

First, **LabUsers** was created to represent the standard users in the lab.

Commands used

```
New-LocalGroup -Name "LabUsers" -Description "Standard users for the lab"
```

```
Get-LocalGroup -Name "LabUsers"
```

The second command was used to verify that the group had been created successfully.

Figure 2 — Creation and verification of the LabUsers local group

```
PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> New-LocalGroup -Name "LabUsers" -Description "Standard users for the lab"

Name      Description
----      -
LabUsers  Standard users for the lab

PS C:\WINDOWS\system32>
PS C:\WINDOWS\system32> Get-LocalGroup -Name "LabUsers"

Name      Description
----      -
LabUsers  Standard users for the lab
```

ProjectTeam

Next, `ProjectTeam` was created for users responsible for project-related resources.

```
New-LocalGroup -Name "ProjectTeam" -Description "Users responsible for project resources"
Get-LocalGroup -Name "ProjectTeam"
```

Figure 3 — Creation and verification of the ProjectTeam local group

```
PS C:\WINDOWS\system32> New-LocalGroup -Name "ProjectTeam" -Description "Users responsible for project resources"

Name      Description
----      -
ProjectTeam Users responsible for project resources

PS C:\WINDOWS\system32> Get-LocalGroup -Name "ProjectTeam"

Name      Description
----      -
ProjectTeam Users responsible for project resources
```

LabAdmins

Finally, `LabAdmins` was created as the administrative group used within the lab scenario.

```
New-LocalGroup -Name "LabAdmins" -Description "IT administrators for the lab"
Get-LocalGroup -Name "LabAdmins"
```

Figure 4 — Creation and verification of the LabAdmins local group

```
PS C:\WINDOWS\system32> New-LocalGroup -Name "LabAdmins" -Description "IT administrators for the lab"

Name      Description
----      -
LabAdmins IT administrators for the lab

PS C:\WINDOWS\system32> Get-Localgroup -Name "LabAdmins"

Name      Description
----      -
LabAdmins IT administrators for the lab
```

Concepts applied

- Creating local groups
- Organizing users according to functions
- Using descriptions to identify the purpose of groups
- Verifying groups using `Get-LocalGroup`

6. Managing Group Memberships

Once the users and groups had been created, each user was added to the appropriate group.

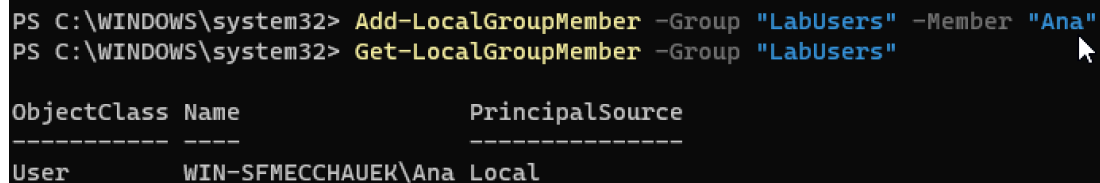
Ana → LabUsers

Ana was added to the LabUsers group.

```
Add-LocalGroupMember -Group "LabUsers" -Member "Ana"  
Get-LocalGroupMember -Group "LabUsers"
```

The second command was used to verify that Ana appeared as a member of the group.

Figure 5 — Adding Ana to the LabUsers group



```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "LabUsers" -Member "Ana"  
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabUsers"
```

ObjectClass	Name	PrincipalSource
User	WIN-SFMECCHAUKEK\Ana	Local

Carlos → ProjectTeam

Next, Carlos was added to the ProjectTeam group.

```
Add-LocalGroupMember -Group "ProjectTeam" -Member "Carlos"  
Get-LocalGroupMember -Group "ProjectTeam"
```

Figure 6 — Adding Carlos to the ProjectTeam group

```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "ProjectTeam" -Member "Carlos"
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "ProjectTeam"
```

ObjectClass	Name	PrincipalSource
User	WIN-SFMECCHAUKE\Carlos	Local

AdminLab → LabAdmins

Finally, AdminLab was added to the LabAdmins group.

```
Add-LocalGroupMember -Group "LabAdmins" -Member "AdminLab"
Get-LocalGroupMember -Group "LabAdmins"
```

Figure 7 — Adding AdminLab to the LabAdmins group

```
PS C:\WINDOWS\system32> Add-LocalGroupMember -Group "LabAdmins" -Member "AdminLab"
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabAdmins"
```

ObjectClass	Name	PrincipalSource
User	WIN-SFMECCHAUKE\AdminLab	Local

It is important to note that belonging to the LabAdmins group does not automatically make AdminLab a Windows administrator. LabAdmins is a group created specifically for this lab and is different from the built-in BUILTIN\Administrators group.

7. General Group Membership Verification

After assigning the users, the configuration of all three groups was verified.

Commands used

```
Get-LocalGroupMember -Group "LabUsers"
Get-LocalGroupMember -Group "ProjectTeam"
```

```
Get-LocalGroupMember -Group "LabAdmins"
```

The resulting configuration was:

```
Ana → LabUsers  
Carlos → ProjectTeam  
AdminLab → LabAdmins
```

This verification confirmed that the user and group structure had been properly prepared before beginning permission management.

Concepts applied

- Group membership
- User administration through groups
- Membership verification
- Separation between users, groups, and permissions

Figure 8 — General verification of group memberships

```
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabUsers"  
  
ObjectClass Name                PrincipalSource  
-----  
User          WIN-SFMECCHAUKEK\Ana Local  
  
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "ProjectTeam"  
  
ObjectClass Name                PrincipalSource  
-----  
User          WIN-SFMECCHAUKEK\Carlos Local  
  
PS C:\WINDOWS\system32> Get-LocalGroupMember -Group "LabAdmins"  
  
ObjectClass Name                PrincipalSource  
-----  
User          WIN-SFMECCHAUKEK\AdminLab Local
```

8. Creating the Resource Structure

To apply and analyze NTFS permissions, a directory structure representing different organizational resources was created.

The structure used was:

```
C:\CompanyLab
├── Public
├── Documents
└── Projects
    ├── ProjectA
    └── ProjectB
```

The directories were created using `New-Item`.

Commands used

```
New-Item -Path "C:\CompanyLab" -ItemType Directory
New-Item -Path "C:\CompanyLab\Public" -ItemType Directory
New-Item -Path "C:\CompanyLab\Documents" -ItemType Directory
New-Item -Path "C:\CompanyLab\Projects" -ItemType Directory
New-Item -Path "C:\CompanyLab\Projects\ProjectA" -ItemType
Directory
New-Item -Path "C:\CompanyLab\Projects\ProjectB" -ItemType
Directory
```

The complete structure was then verified using:

```
Get-ChildItem "C:\CompanyLab" -Recurse
```

The `-Recurse` parameter made it possible to also display the directories located inside `Projects`.

Concepts applied

- Creating directories
- Hierarchical organization of resources
- Using paths
- Recursive verification of a directory structure

Figure 9 — Verification of the CompanyLab directory structure

```
PS C:\WINDOWS\system32> Get-ChildItem "C:\CompanyLab" -Recurse

Directory: C:\CompanyLab

Mode                LastWriteTime         Length Name
----                -
d-----         8/11/2026  12:13 PM              Documents
d-----         8/11/2026  12:15 PM              Projects
d-----         8/11/2026  12:11 PM              Public

Directory: C:\CompanyLab\Projects

Mode                LastWriteTime         Length Name
----                -
d-----         8/11/2026  12:15 PM              ProjectA
d-----         8/11/2026  12:15 PM              ProjectB
```

9. Inspecting the Default ACL

Before making any permission changes, the Access Control List for CompanyLab was inspected.

Initially, the following command was used:

```
Get-Acl "C:\CompanyLab"
```

The output provided general information such as the owner of the directory and its access entries.

To display the individual ACL entries, the following command was used:

```
(Get-Acl "C:\CompanyLab").Access
```

The inspection showed several identities configured by Windows, including:

- BUILTIN\Administrators
- NT AUTHORITY\SYSTEM
- BUILTIN\Users
- NT AUTHORITY\Authenticated Users

Properties such as the following were also analyzed:

- `FileSystemRights`
- `AccessControlType`
- `IdentityReference`
- `IsInherited`
- `InheritanceFlags`
- `PropagationFlags`

For example, `BUILTIN\Administrators` had `FullControl`, while `BUILTIN\Users` had more limited permissions such as `ReadAndExecute`.

The `IsInherited = True` property made it possible to identify that certain entries had been inherited from a parent directory.

Indicators related to permission propagation were also observed:

- `ObjectInherit` — inheritance to files.
- `ContainerInherit` — inheritance to subdirectories.
- `InheritOnly` — an entry intended to be inherited by descendant objects.

Concepts applied

- ACL (Access Control List)
- DACL (Discretionary Access Control List)
- Access control entries
- `Allow`
- `FullControl`
- `ReadAndExecute`
- Inherited permissions
- `ObjectInherit`
- `ContainerInherit`
- `InheritOnly`

Figure 10 — Inspection of the CompanyLab ACL access entries

```
PS C:\WINDOWS\system32> (Get-Acl "C:\CompanyLab").Access

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : BUILTIN\Administrators
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : FullControl
AccessControlType : Allow
IdentityReference : NT AUTHORITY\SYSTEM
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : ReadAndExecute, Synchronize
AccessControlType : Allow
IdentityReference : BUILTIN\Users
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : None

FileSystemRights : Modify, Synchronize
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : True
InheritanceFlags : None
PropagationFlags : None

FileSystemRights : -536805376
AccessControlType : Allow
IdentityReference : NT AUTHORITY\Authenticated Users
IsInherited : True
InheritanceFlags : ContainerInherit, ObjectInherit
PropagationFlags : InheritOnly
```

10. Configuring Permissions for LabUsers

After inspecting the existing permissions, access was configured for the groups created during the lab.

The LabUsers group was granted Read & Execute permission on the Public directory.

Command used

```
icaccls "C:\CompanyLab\Public" /grant "LabUsers:(OI)(CI)RX"
```

In this configuration:

- `/grant` grants a permission.
- `LabUsers` identifies the group receiving the permission.
- `RX` represents Read & Execute.
- `OI` represents Object Inherit.
- `CI` represents Container Inherit.

Therefore, `LabUsers` received Read & Execute permission on `Public`, with the ability for the permission to propagate to files and subdirectories.

The configuration was verified using:

```
icacls "C:\CompanyLab\Public"
```

The ACL was also inspected using:

```
(Get-Acl "C:\CompanyLab\Public").Access
```

This confirmed the presence of `LabUsers` with the corresponding permission.

Concepts applied

- Assigning NTFS permissions
- Read & Execute
- Using `/grant`
- Object Inherit
- Container Inherit
- Permission verification

Figure 11 — Assigning and verifying Read & Execute for LabUsers on Public

```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Public" /grant "LabUsers:(OI)(CI)RX"
processed file: C:\CompanyLab\Public
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Public"
C:\CompanyLab\Public WIN-SFMECCHAUKE\LabUsers:(OI)(CI)(RX)
                     BUILTIN\Administrators:(I)(OI)(CI)(F)
                     NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                     BUILTIN\Users:(I)(OI)(CI)(RX)
                     NT AUTHORITY\Authenticated Users:(I)(M)
                     NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

11. Configuring Permissions for ProjectTeam

The `ProjectTeam` group was granted `Modify` permission on the `Projects` directory.

Command used

```
icaccls "C:\CompanyLab\Projects" /grant "ProjectTeam:(OI)(CI)M"
```

In this configuration:

- `M` represents `Modify`.
- `OI` allows inheritance to files.
- `CI` allows inheritance to subdirectories.

The `Modify` permission provides a higher level of access than `Read & Execute`, allowing changes to be made to the content according to the configured access rules.

The configuration was verified using:

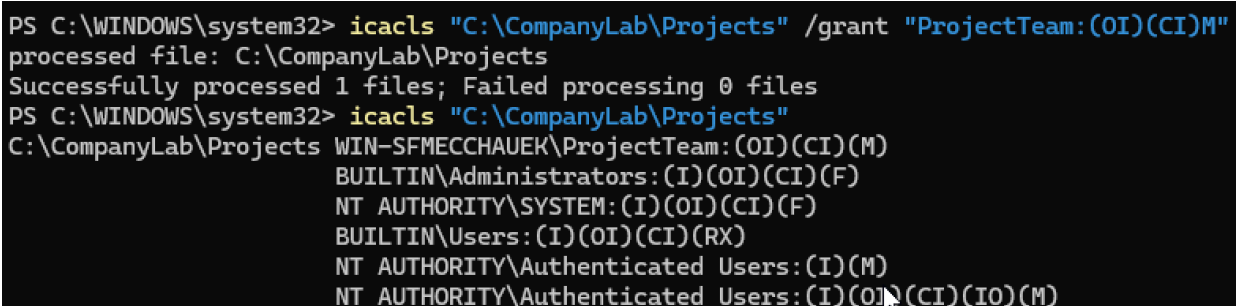
```
icaccls "C:\CompanyLab\Projects"
```

The output confirmed that `ProjectTeam` had the configured permission on the directory.

Concepts applied

- `Modify` permission
- Assigning permissions to groups
- Inheritance to files
- Inheritance to subdirectories
- Verification using `icaccls`

Figure 12 — Assigning and verifying `Modify` for `ProjectTeam` on `Projects`



```
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Projects" /grant "ProjectTeam:(OI)(CI)M"
processed file: C:\CompanyLab\Projects
Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icaccls "C:\CompanyLab\Projects"
C:\CompanyLab\Projects  WIN-SFMECCHAUKE\ProjectTeam:(OI)(CI)(M)
                        BUILTIN\Administrators:(I)(OI)(CI)(F)
                        NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                        BUILTIN\Users:(I)(OI)(CI)(RX)
                        NT AUTHORITY\Authenticated Users:(I)(M)
                        NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

12. Configuring Permissions for LabAdmins

To represent a group with a higher level of access to the lab resources, **LabAdmins** was granted **Full Control** over **CompanyLab**.

Command used

```
icacls "C:\CompanyLab" /grant "LabAdmins:(OI)(CI)F"
```

In this configuration:

- F represents Full Control.
- OI allows inheritance to files.
- CI allows inheritance to subdirectories.

The configuration was then verified using:

```
icacls "C:\CompanyLab"
```

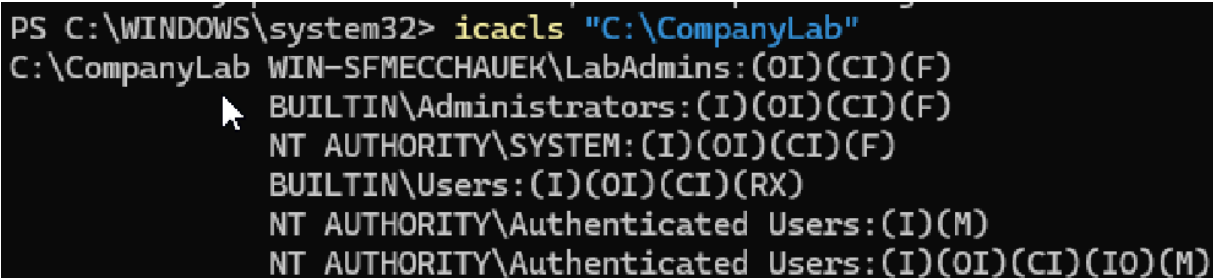
This confirmed that **LabAdmins** had received Full Control over the resource.

The **LabAdmins** group should be distinguished from **BUILTIN\Administrators**. **LabAdmins** was created specifically during this lab, and its capabilities depend on the permissions assigned to it.

Concepts applied

- Full Control
- Permission management through groups
- Permission inheritance
- Difference between an administrator-created group and Windows built-in groups

Figure 13 — Assigning and verifying Full Control for LabAdmins on CompanyLab



```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab"
C:\CompanyLab WIN-SFMECCHAUKE\LabAdmins:(OI)(CI)(F)
                BUILTIN\Administrators:(I)(OI)(CI)(F)
                NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                BUILTIN\Users:(I)(OI)(CI)(RX)
                NT AUTHORITY\Authenticated Users:(I)(M)
                NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

13. Verifying Permission Inheritance

After granting **Modify** to **ProjectTeam** on **Projects**, it was verified whether the permission had propagated to one of its subdirectories.

The relevant structure was:

```
Projects
├── ProjectA
└── ProjectB
```

The permission had been configured on **Projects** using:

```
(OI)(CI)M
```

No permission was assigned directly to **ProjectA**.

To inspect its ACL, the following command was used:

```
icacls "C:\CompanyLab\Projects\ProjectA"
```

The entry corresponding to **ProjectTeam** displayed the following indicator:

```
(I)
```

(I) represents **Inherited**, indicating that the permission had not been configured directly on **ProjectA**, but had instead been inherited from its parent directory, **Projects**.

The observed relationship was:

```
ProjectTeam
  |
  | Modify
  ↓
Projects
  |
  | inheritance
  ↓
ProjectA
  └── Modify (Inherited)
```

Since Carlos belongs to `ProjectTeam`, the access model used during the lab can be represented as:

```
Carlos
  ↓
ProjectTeam
  ↓
Modify
  ↓
Projects
  ↓
ProjectA
```

Concepts applied

- NTFS permission inheritance
- Inherited permissions
- (I) indicator
- Object Inherit
- Container Inherit
- Permission propagation between directories

Figure 14 — Verification of ProjectTeam's inherited permission on ProjectA

```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Projects\ProjectA"
C:\CompanyLab\Projects\ProjectA WIN-SFMECCHAUKE\ProjectTeam:(I)(OI)(CI)(M)
                                WIN-SFMECCHAUKE\LabAdmins:(I)(OI)(CI)(F)
                                BUILTIN\Administrators:(I)(OI)(CI)(F)
                                NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                                BUILTIN\Users:(I)(OI)(CI)(RX)
                                NT AUTHORITY\Authenticated Users:(I)(M)
                                NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

14. Final Permission Verification

As the final step, the permissions configured on the main lab resources were verified together.

Commands used

```
icacls "C:\CompanyLab\Public"
icacls "C:\CompanyLab\Projects"
icacls "C:\CompanyLab"
```

The main resulting configuration was:

- LabUsers → Public → Read & Execute (RX)
- ProjectTeam → Projects → Modify (M)
- LabAdmins → CompanyLab → Full Control (F)

This verification confirmed that the three access levels configured during the lab remained correctly assigned.

Figure 15 — Final verification of the configured permissions

```
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Public"
C:\CompanyLab\Public WIN-SFMECCHAUKE\LabUsers:(OI)(CI)(RX)
                     WIN-SFMECCHAUKE\LabAdmins:(I)(OI)(CI)(F)
                     BUILTIN\Administrators:(I)(OI)(CI)(F)
                     NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                     BUILTIN\Users:(I)(OI)(CI)(RX)
                     NT AUTHORITY\Authenticated Users:(I)(M)
                     NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls "C:\CompanyLab\Projects"
C:\CompanyLab\Projects WIN-SFMECCHAUKE\ProjectTeam:(OI)(CI)(M)
                      WIN-SFMECCHAUKE\LabAdmins:(I)(OI)(CI)(F)
                      BUILTIN\Administrators:(I)(OI)(CI)(F)
                      NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
                      BUILTIN\Users:(I)(OI)(CI)(RX)
                      NT AUTHORITY\Authenticated Users:(I)(M)
                      NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)

Successfully processed 1 files; Failed processing 0 files
PS C:\WINDOWS\system32> icacls "C:\CompanyLab"
C:\CompanyLab WIN-SFMECCHAUKE\LabAdmins:(OI)(CI)(F)
              BUILTIN\Administrators:(I)(OI)(CI)(F)
              NT AUTHORITY\SYSTEM:(I)(OI)(CI)(F)
              BUILTIN\Users:(I)(OI)(CI)(RX)
              NT AUTHORITY\Authenticated Users:(I)(M)
              NT AUTHORITY\Authenticated Users:(I)(OI)(CI)(IO)(M)
```

15. Tests Performed

The following tasks were successfully validated during the lab:

- Creation of local users.
- Verification of account status.
- Creation of local groups.
- Addition of users to groups.
- Individual and general verification of group memberships.
- Creation of a hierarchical resource structure.
- Inspection of existing ACLs before making changes.

- Identification of default Windows permissions.
- Assignment of Read & Execute to **LabUsers**.
- Assignment of Modify to **ProjectTeam**.
- Assignment of Full Control to **LabAdmins**.
- Configuration of inheritance using **OI** and **CI**.
- Identification of inherited permissions using **(I)**.
- Final verification of permissions using **icacls**.

16. Analysis

The lab provided an understanding of how Windows uses users, groups, and Access Control Lists to manage access to file system resources.

One of the main concepts was the separation between identity and authorization. An account can be enabled without having administrative privileges, while access to specific resources depends on group memberships and the configured ACLs.

Using groups made it possible to implement an organized administration model. Instead of assigning all permissions directly to individual users, **LabUsers**, **ProjectTeam**, and **LabAdmins** were used as groups to which different levels of access were granted.

The differences between Read & Execute, Modify, and Full Control were also observed, making it possible to associate different permission levels with different functions within the scenario.

Inspecting the ACL before making changes showed that Windows already maintains access entries for identities such as **Administrators**, **SYSTEM**, **Users**, and **Authenticated Users**. This made it possible to distinguish existing system permissions from those specifically added during the lab.

Finally, the verification performed on **ProjectA** demonstrated how inheritance works in practice. The **Modify** permission granted to **ProjectTeam** on **Projects** propagated to the subdirectory and was identified by the **(I)** indicator.

17. Conclusion

This lab provided practical experience with fundamental concepts of user, group, and permission management in Windows using PowerShell.

A practical scenario was built in which different accounts were organized into groups, and different levels of access were subsequently assigned to NTFS file system resources.

Get-Acl was used to inspect existing access control lists, while **icacls** was used to configure and verify permissions such as Read & Execute, Modify, and Full Control.

The lab also demonstrated how permission inheritance works between directories, providing an understanding of how a configuration applied to a parent resource can propagate to its descendant objects.

These concepts provide an important foundation for technical support and Windows system administration tasks, where proper management of users, groups, and permissions is essential for controlling access to resources.